PDF RAG Chatbot — Project Documentation

---

1. Goal

Build a simple RAG (Retrieval-Augmented Generation) system as a first, small project before tackling something bigger. The idea: upload one PDF, ask questions about it, get answers grounded only in that document — not the model's general knowledge.

---

2. Architecture

```
PDF  →  text extraction  →  chunking  →  embeddings  →  FAISS index
                                                              |
question  →  embed question  →  search index  →  top-k chunks
                                                              |
                                        chunks + question → Gemini → answer
```

Three scripts, split by responsibility:

- **`ingest.py`** — one-time processing of the PDF into a searchable index.
- **`query.py`** — takes a question, retrieves relevant chunks, calls the LLM to generate an answer.
- **`app.py`** — Gradio UI wiring the two together (upload + chat).

---

3. Key decisions and why

Interface: Gradio, not Streamlit

Started with Streamlit as the default choice, but switched to Gradio because it has a built-in `ChatInterface` component purpose-built for chat — much less boilerplate than manually looping over `st.chat_message` in Streamlit. Still just one Python file, still free to deploy (e.g. Hugging Face Spaces).

Vector store: FAISS, not Postgres/pgvector

FAISS is in-memory and needs no server — just a pip install. Good fit for a "simple, single-PDF" project. Already know Postgres/pgvector from other work, so that's a natural upgrade path for a bigger/multi-document version later, not needed here.

Embeddings: sentence-transformers (local), not an API

Used `all-MiniLM-L6-v2` — runs on CPU, no GPU needed, no per-call cost or API key required just to embed text. Keeps the "retrieval" half of RAG completely free and offline; only the "generation" half needs an API.

LLM for generation: switched from Claude API to Gemini API

Originally built with the Anthropic API since that's the model I use day-to-day. Switched to Google's Gemini API (`gemini-2.0-flash`) by request — free tier, and decouples this project from needing an Anthropic key specifically. The swap only touched `query.py` (the retrieval/embedding logic didn't change at all) — a good sign the retrieval and generation layers were properly separated from the start.

Chunking: simple character-based sliding window

`CHUNK_SIZE = 800` characters, `CHUNK_OVERLAP = 100`. Deliberately basic (not sentence/paragraph-aware) to keep the first version simple. Known limitation: can cut a chunk mid-sentence or split a bulleted list across two chunks — this came back to matter later (see Section 4).

---

4. TOP_K and the retrieval failure

`TOP_K` = how many chunks get pulled to answer a question. Tested it with 5 questions (see `eval_questions.md`). 4 were answered correctly. The 5th one asked for "two advantages" — the bot gave one correct advantage and one wrong one, because the actual second advantage was in a chunk that didn't

get retrieved. Instead of saying “I don’t have that,” it grabbed a different true sentence from elsewhere in the PDF and used it as if it answered the question — so it looked right, but wasn’t.

The big lesson: A RAG bot can sound completely confident and still be wrong — not because it made something up, but because it pulled a real fact from the wrong place in the document. That’s a sneakier kind of error than obvious hallucination, and it’s exactly why testing with known correct answers (`eval_questions.md`) is valuable — you’d never catch this just by reading the answer and thinking “sounds right.”

---

5. Evaluation approach

Built `eval_questions.md` — 5 hand-picked questions from the source PDF, each with a ground-truth answer written by hand from the actual document text (not generated). Deliberately included two “precise fact” questions (a specific ratio, a specific rule) and one “spans a bulleted list” question, since those are the categories most likely to expose chunking/retrieval problems rather than pure LLM reasoning problems.

Lesson: for RAG specifically, evaluation questions are more useful if chosen to stress *retrieval*, not just the LLM’s ability to summarize text it’s already been handed. A model can be excellent at generation and still give wrong answers if the retrieval step feeds it the wrong chunks.

---

6. Version control

Set up git locally, added a `.gitignore` to keep `venv/`, the generated `index/` folder, and any API keys out of the repository — all of those are either machine-specific, regenerable, or secret, and none belong in version control.